

SWE4004 · CLOUD COMPUTING AND APPLICATIONS

# RoadAssist Bharat

AI-Powered · Cloud-Connected · Network-Resilient  
Emergency Mobility Platform

"RoadAssist doesn't stop when the network stops."

## TEAM

V Saatwik Sairaam · P Sai Nirisha Chowdary

T V S Jignesh · G Parthavi

## FACULTY

Dr. Nagendra Panini Challa

## INSTITUTION

VIT-AP University

## THE PROBLEM

# A breakdown is not a software problem

...until you realise the app you would use to fix it needs the network you do not have.

21:40

A Swift stops on NH-48. Two hundred kilometres from home, one bar of signal.

21:41

The driver opens a roadside-assistance app. It shows a spinner.

21:44

Still a spinner. There is no way to tell whether anything was sent.

21:52

They give up and start calling numbers from a sticker on the windscreen.

## WHAT ACTUALLY FAILS

- Coverage is worst exactly where breakdowns are most dangerous
- No way to know if a request was sent
- No centralised incident record
- Response time is unmeasured, so it cannot be improved

# One platform, from incident to resolution



Every step is real application state. No manual database editing anywhere in the normal workflow.

# Six things, and one that matters most

## AI-ASSISTED DIAGNOSIS

Rules engine, explainable,  
same table on device and cloud

## DYNAMIC DISPATCH

Ranked, wave-based,  
busy providers excluded

## REAL-TIME TRACKING

Server-sent events,  
measured at 65 ms

## SECURE PAYMENTS

Server-side amount,  
signature-verified webhook

## INCIDENT INTELLIGENCE

Severity, capability,  
tamper-evident audit

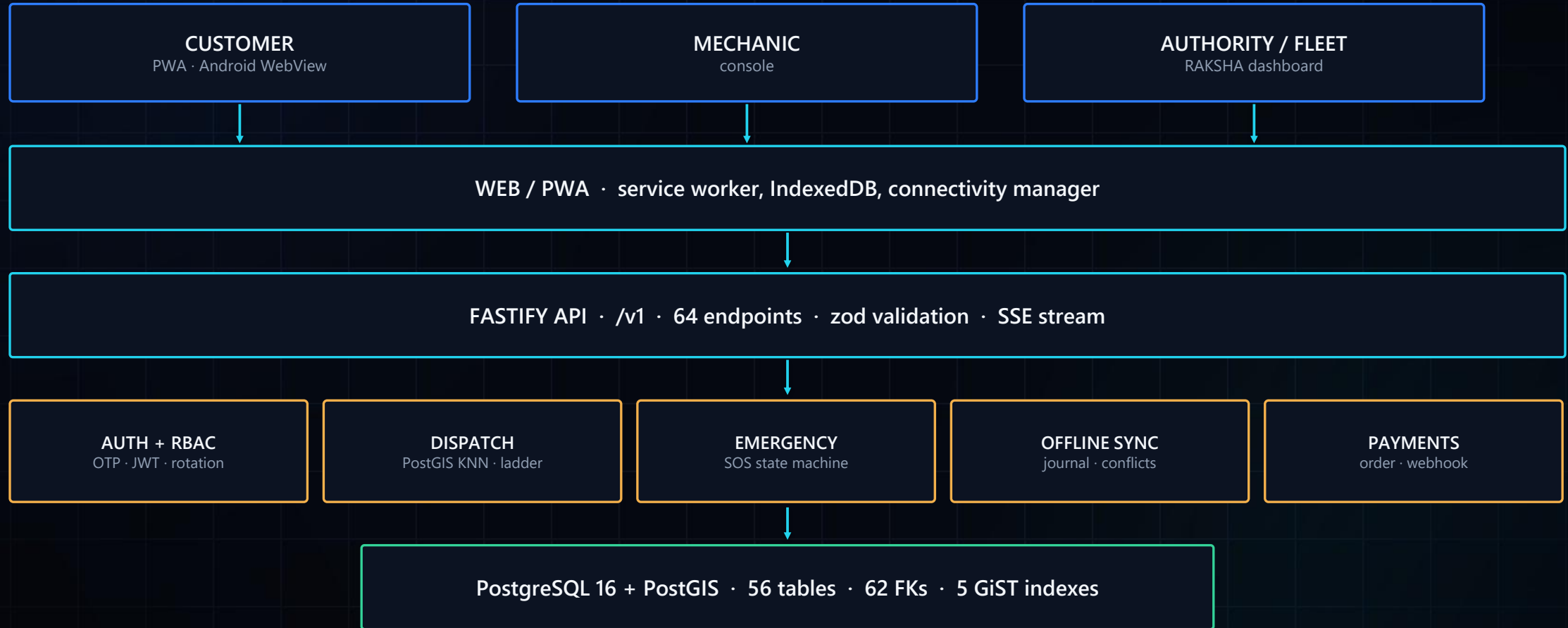
## OFFLINE-FIRST EMERGENCY

SOS with no network,  
stored and forwarded

“RoadAssist doesn't stop when the network stops.”

# System architecture

One container serving three surfaces, one PostGIS database. Everything external is behind an adapter.



# Two paths, and the second one is the product



The connectivity manager decides which path runs — ONLINE, LIMITED or OFF-GRID — from measured evidence, never from navigator.onLine alone.

# Off-grid SOS

No internet does not mean the emergency workflow disappears. It means it runs here instead.



## WHAT THE USER IS TOLD

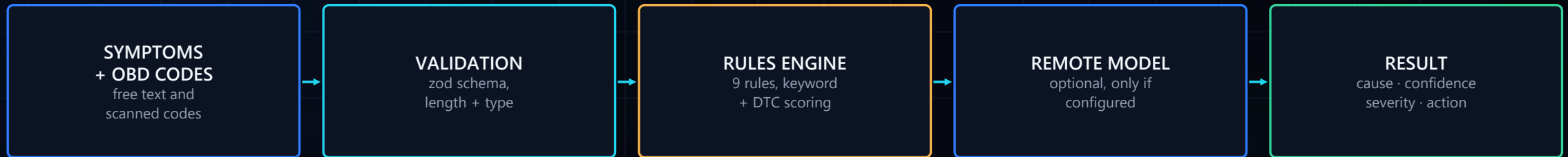
### OFF-GRID SOS CREATED

No network connection detected.

Your emergency information is stored securely on this device and will be synchronized automatically when connectivity returns.

"Nothing has been transmitted."  
We never claim emergency services were contacted. The 112 handoff is a stub and the API says so.

# Diagnosis: rules first, model optional



## THE SAFETY ASYMMETRY

A model may make a drivability verdict STRICTER. It can never make one laxer.

The rules result is always computed. A remote answer is used only if it clears AI\_MIN\_CONFIDENCE and does not contradict the rules on safety.

## STATED PLAINLY

This is a deterministic rules engine, not machine learning. The UI labels it "rules-1.0.0", and "LOCAL OFFLINE DIAGNOSIS" on the device.

A trained model DOES exist in the project — YOLO11n for road damage, mAP50 0.443 — and those metrics are measured.

EXPLAINABLE, NOT OPAQUE

# Incident intelligence

Every field below is derived from a rule you can read, not from a weighting nobody can inspect.



## EXPLAINABLE

“Battery discharged or terminals loose” — because these keywords matched, at this confidence

## AUDITABLE

Every prediction writes a model\_predictions row: capability, version, confidence, latency, input HASH (never the text)

## HONEST

Below 55% confidence the UI says so: “At 42% this is a suggestion, not a finding”

## NEVER FABRICATED

No ETA unless a provider is travelling. No location unless GPS returned one.

# Dispatch is dynamic scheduling

Work assigned at run time from a pool, by a score computed from live state, with timeout-driven re-scheduling.



## THE BUG THIS AUDIT FOUND

Dispatch filtered on `is_available` alone — a duty toggle meaning “I am working today”, not “I am free now”. A mechanic already driving to one breakdown kept receiving offers for the next.

Fixed: provider state is DERIVED from live data.

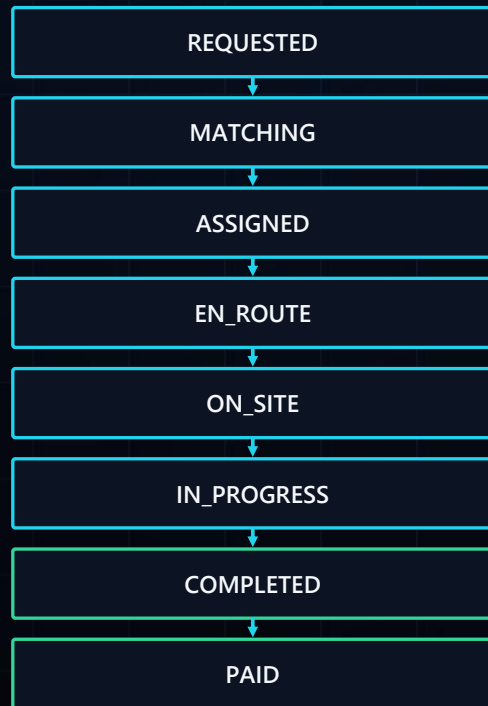
## PROVIDER STATES

|                          |                                    |
|--------------------------|------------------------------------|
| OFFLINE                  | off duty — never offered           |
| AVAILABLE                | free — offerable                   |
| OFFERED                  | holding an offer — still offerable |
| BUSY / EN_ROUTE / ON_JOB | committed — never offered          |

# Two guarded state machines

A client sends a command; only the transition table decides whether it is legal. Every change writes a timestamped event row.

**BOOKING — 13 states, 15 commands**    **EMERGENCY — ADR-0005 in code**



## THE RULE THAT IS ENFORCED

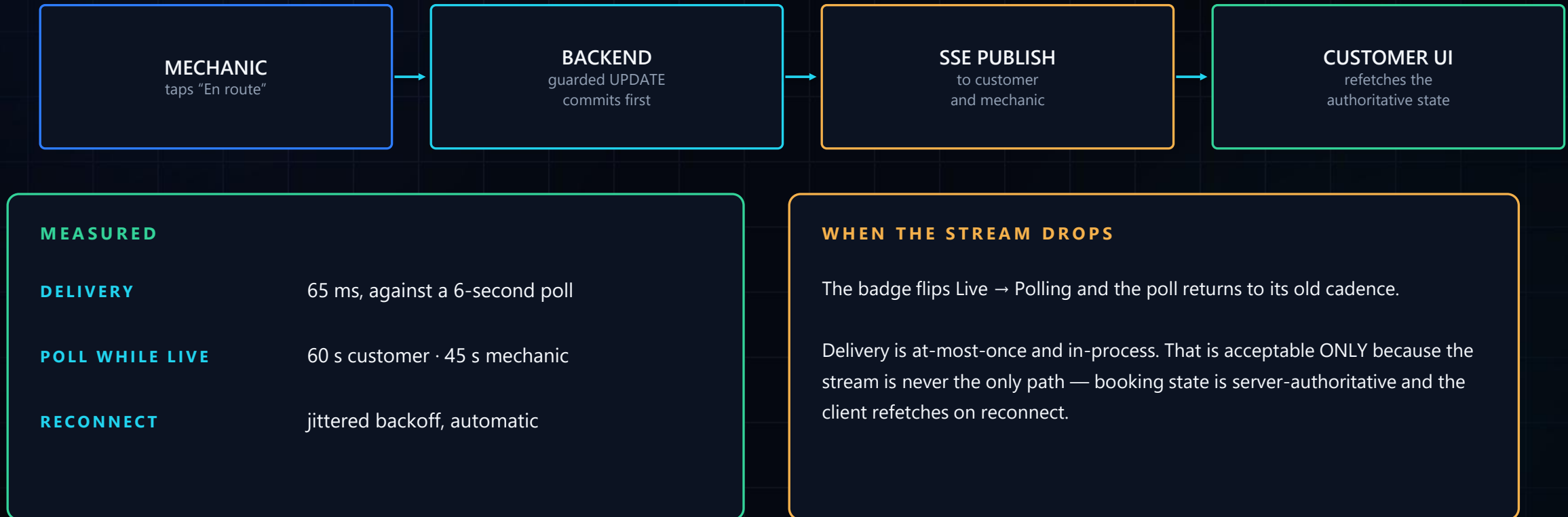
A model raises a SIGNAL. A human confirms an INCIDENT.

“escalate” is reachable only from CONFIRMED. There is no path from a model-detected crash to a dispatched responder that does not pass through a person.

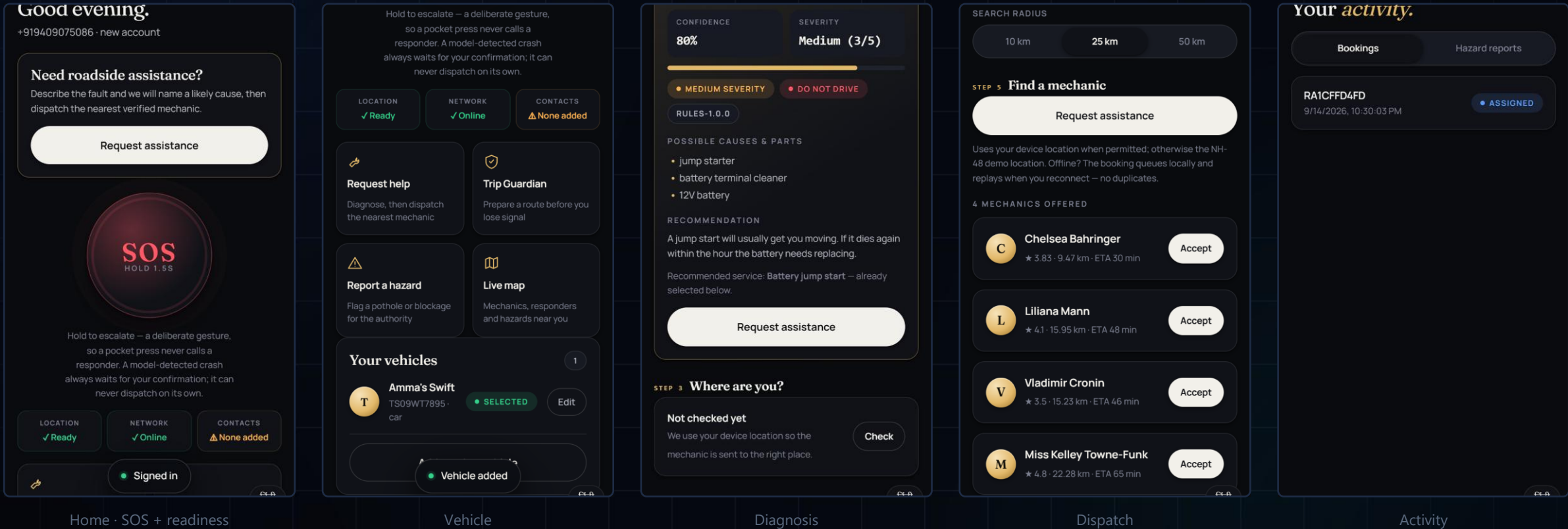
That is one line in a transition table, and it is unit-tested.

# Real-time, but never the source of truth

Server-sent events over one long-lived GET. Persist first, publish second — always.

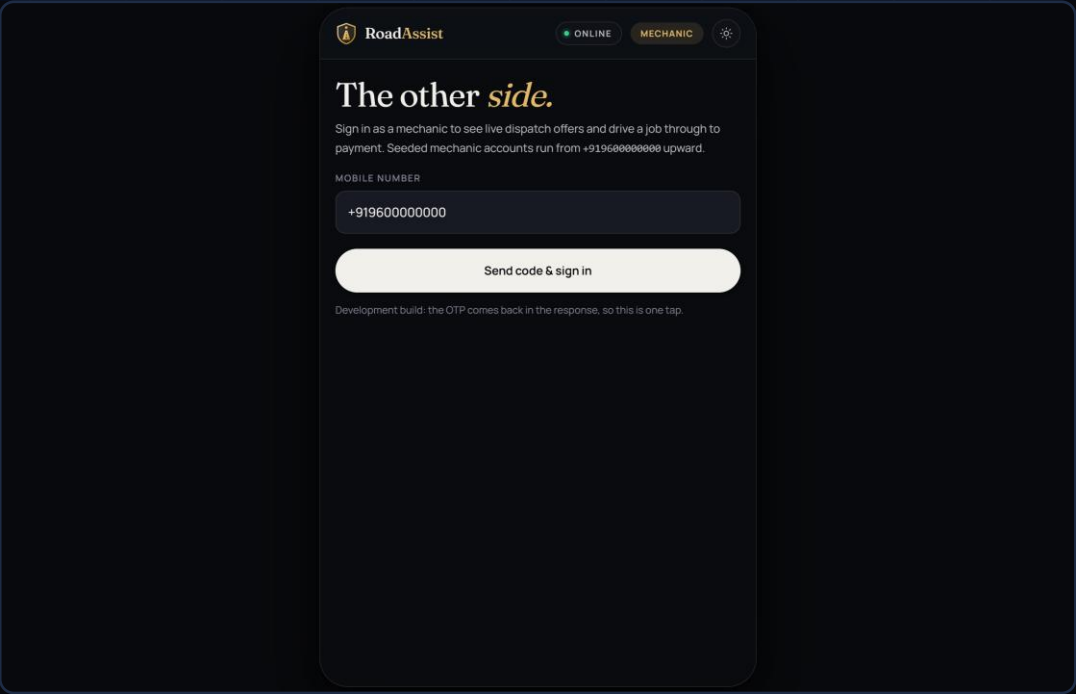


# Customer application

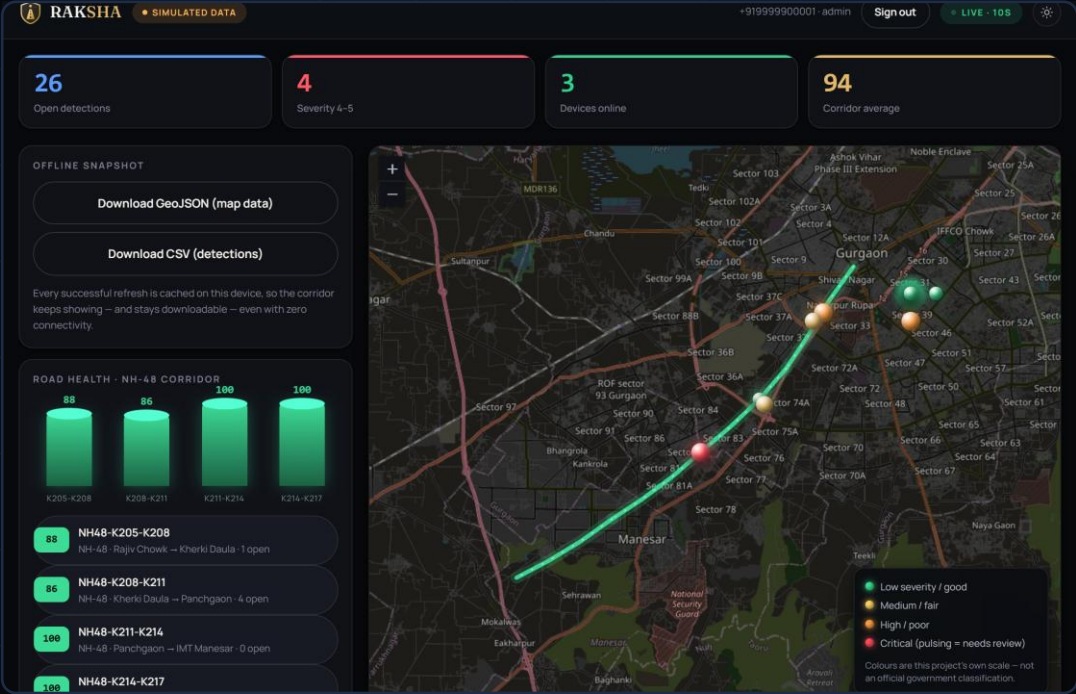


Captured by scripts/capture-screens.mjs against the live API — regenerable in ninety seconds, so they cannot drift from the product.

# Mechanic console and authority dashboard



Mechanic console · dispatch inbox, live badge, job lifecycle  
Mechanic: availability · incoming request · accept · decline · en route · arrived · in service · complete — every transition persisted and pushed to the customer.



# Three states, three honest answers

## ONLINE

- Everything works
- SOS reaches dispatch
- Live tracking + ETA
- Cloud diagnosis

## LIMITED

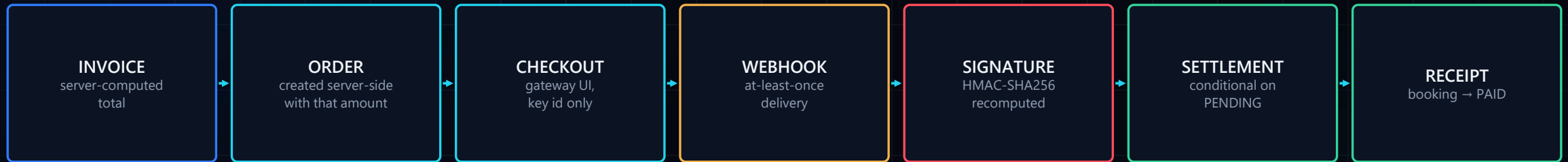
- Requests still attempted
- Anything that fails is STORED
- Slow or flaky path
- User is told it is weak

## OFF-GRID

- SOS stored on device
- GPS still works
- Local diagnosis
- Cached maps · no live anything

The indicator names the state in WORDS as well as colour — status is never carried by tint alone.  
Under every state the primary action stays the same and stays one tap away: SOS.

# The client never decides that money arrived



## AMOUNT

Never taken from the request — it is the invoice total, and the webhook refuses a mismatch

## TWO PATHS

Browser callback AND webhook. The webhook is the one that matters: a customer can pay and close the tab

## IDEMPOTENT

Delivery is at-least-once, so a replayed webhook settles nothing twice

## REFUSES TO BOOT

Production will not start on a real gateway with no webhook secret

## TESTED

22 assertions against a local stub of Razorpay's API: forged signature, replay, wrong amount, wrong order — each fails closed

# Security

The suite passes when the platform refuses — “we tried and could not get in” is a stronger statement than “the code looks right”.

## AUTHENTICATION

OTP → JWT, rotating refresh  
with reuse detection

## RBAC

role gates on every  
operator surface

## RESOURCE OWNERSHIP

checked at the resource,  
not just the route

## RATE LIMITING

per principal — and never  
on SOS escalation

## INPUT VALIDATION

zod at every boundary,  
parameterised SQL

## PAYMENT VERIFICATION

server-side amount,  
signed webhook

## AUDIT LOG

hash-chained, append-only,  
verified live

74 / 74

attacks refused

Sensitive data: medical break-glass needs a role, a LIVE incident and a written reason, and writes a tamper-evident row.

Logs redact credentials, OTP codes, phone numbers, medical fields and coordinates — and that redaction is unit-tested.

Three real vulnerabilities were found by this suite during the audit and fixed: missing request ids on 401s, a 500 on malformed JSON, an unsigned SMS intake that did not say so.

# PostgreSQL 16 + PostGIS

Chosen for one query, and kept for four more reasons.

56

tables

62

foreign keys

137

indexes

5

GiST spatial

5

check constraints

5

migrations

## WHY POSTGIS

“nearest available mechanic to this point” is ONE indexed query with ST\_DWithin doing true metres-on-a-sphere. Without it: fetch every mechanic and compute haversine in app code.

## WHY RELATIONAL

SELECT ... FOR UPDATE is what stops two mechanics taking one job. Transactions are the feature, not a formality.

## CONSTRAINTS

client\_incident\_id UNIQUE is the entire duplicate-emergency defence. Money is integer paise; ratings are CHECK 1–5.

## AUDITABILITY

audit\_log is hash-chained and Postgres RULES block UPDATE and DELETE — an admin cannot quietly edit it.

## VERIFIED

Migrations run from an EMPTY database in this audit, then the full suite. Backup + restore rehearsed: chain intact across 639 entries.

# Where we sit in the cloud

We are a consumer of IaaS and PaaS, and a provider of SaaS to three user classes.

## SaaS

We PROVIDE this: citizen app,  
mechanic console, authority dashboard

IMPLEMENTED

## PaaS

We would CONSUME managed  
Postgres + container platform

TARGET

## IaaS

Compute, network, storage.  
Compose is the local stand-in

PARTIAL

### CHARACTERISTICS WE ACTUALLY EXHIBIT

Broad network access — one API serves a browser, an installed PWA, an Android WebView and a feature phone over SMS.  
Resource pooling — dispatch pools providers and allocates on demand.

### DEPLOYMENT MODEL

Public cloud, Indian region. The constraint that decides it is regulatory, not technical: “No PII leaves India” — including logs, backups and crash reports.  
Status: designed, not deployed.

## MODULE 2

# Enabling technologies

Five topics. Four are genuinely built; one is partial and says so.

### Data centre

Compose stack: PostGIS + Redis + Redpanda on a private network; DB port NOT published in production

PARTIAL

### Virtualization

Multi-stage container, non-root (uid 1000), tini as PID 1, healthcheck. Built and run — 321 MB.

IMPLEMENTED

### Web technology

REST over HTTPS with a uniform envelope, server-sent events for real-time, PWA with a service worker

IMPLEMENTED

### Multitenancy

Row-scoped shared schema. 12 cross-tenant attacks fired; all 12 refused.

IMPLEMENTED

### Service technology

Versioned /v1, 64 endpoints, zod validation, stable error codes, idempotency keys

IMPLEMENTED

## MODULE 3

# Infrastructure mechanisms

|                        |                                                                                                                |             |
|------------------------|----------------------------------------------------------------------------------------------------------------|-------------|
| Virtual server         | Container image; runs anywhere Docker runs. Full test suite passes AGAINST the image.                          | IMPLEMENTED |
| Cloud storage device   | Postgres volume, upload volume, and client-side IndexedDB with AES-GCM-256 at rest                             | IMPLEMENTED |
| Cloud usage monitor    | Structured JSON logs, per-request correlation id, operation duration/result, /v1/ops/overview with live counts | PARTIAL     |
| Ready-made environment | compose → migrate → seed → start. Reference-only seed for production verified: 0 demo rows.                    | IMPLEMENTED |
| Network perimeter      | DB has no published port; API on loopback behind a proxy; CORS allowlist enforced in production                | PARTIAL     |
| Resource replication   | Stateless API is the precondition and is met. Nothing is replicated.                                           | TARGET      |

## MODULE 4

# The eight fundamental architectures

Two are built. Six are design. I will not show you an animation and call it an autoscaler.

### Workload distribution

The dispatch engine distributes jobs across a provider pool by computed score

IMPLEMENTED

### Resource pooling

Providers are a pool; state is derived and busy members excluded

IMPLEMENTED

### Dynamic scalability

HPA on queue depth. BUILT: the readiness gate — /health returns 503 when the DB is down

TARGET

### Elastic resource capacity

Cluster autoscaler under the pods

TARGET

### Service load balancing

LB with health-checked rotation

TARGET

### Cloud bursting

Modelled evening scenario — labelled as modelled

TARGET

### Elastic disk provisioning

Storage allocated as data arrives

TARGET

### Redundant storage

Replica with failover. Backup procedure documented and rehearsed.

TARGET

Three things block replication today, and each is documented where it is defined: the SSE registry, the rate limiter and the offer sweeper are in-process.

The fix for each is already designed — Redis, and the outbox → Redpanda bus already in the architecture.

# Static vs dynamic scheduling

## STATIC

Decided before run time, on a fixed schedule.

- Offer sweeper — every 10 s (OFFER\_SWEEP\_SECONDS)
- Client booking poll — 6 s, or 60 s while streaming
- SSE heartbeat — every 25 s
- Retention purge — 24 h after a sync

## DYNAMIC

Decided at run time, from live state.

- Dispatch: pool queried at request time
- Score from live distance, rating, workload
- Wave offered; timeout re-schedules to the next wave
- A decline escalates immediately
- Assignment settled under a row lock — exactly one

**RoadAssist's dispatch engine IS dynamic scheduling — it is the clearest Module 4 mechanism the project actually implements.**

## TESTING

# 631 assertions, all executed

Against a real PostgreSQL + PostGIS and a real Chrome. Every number here is from a run, not an estimate.

|                         |     |                                                                        |
|-------------------------|-----|------------------------------------------------------------------------|
| Unit                    | 101 | state machines, rules, redaction, SMS coordinates, i18n segment budget |
| End-to-end              | 189 | the whole API journey against real Postgres                            |
| Concurrency + real-time | 65  | paces a sequential suite structurally cannot make                      |
| Security                | 74  | attacks that must FAIL                                                 |
| Gateway security        | 26  | webhook signatures, append-only rules, OTP ceilings                    |
| Payment sandbox         | 22  | forged signature, replay, wrong amount — fails closed                  |
| Browser / offline       | 163 | what only a browser can prove                                          |

**600 passed · 0 failed · 0 skipped · coverage 95.68% lines on the pure domain modules · 0 server errors across the sweep**

# Every failure has an answer

DETECT → DEGRADE → PRESERVE DATA → RECOVER. Twenty rows in the failure matrix; seven of them here.

| FAILURE             | DETECTED BY                   | USER EXPERIENCE                      | RECOVERY                  |
|---------------------|-------------------------------|--------------------------------------|---------------------------|
| Network lost        | connectivity manager          | off-grid mode; SOS stored locally    | auto on reconnect         |
| Backend unreachable | probe fails, radio up         | LIMITED, then OFF-GRID               | auto                      |
| Database down       | /health → 503                 | safe 500 with a request id           | auto, no restart — tested |
| GPS denied          | geolocation error             | “Unknown — denied”, never a fake fix | next fix                  |
| AI endpoint down    | timeout                       | rules engine — nothing visible       | auto                      |
| Payment fails       | gateway error / bad signature | invoice stays unpaid                 | customer retries          |
| Provider times out  | expires_at passes             | sweeper escalates to next wave       | ladder continues          |

Each row names the suite that proves it. Database loss and backup/restore are now automated CI steps, not prose.

## IMPLEMENTATION STATUS

# What is built, what is partial, what is design

Marked from the code, not from intent.

|                                                                          |             |                                                                                      |             |
|--------------------------------------------------------------------------|-------------|--------------------------------------------------------------------------------------|-------------|
| <b>AI diagnosis</b><br>rules engine + trained YOLO11n for road damage    | PARTIAL     | <b>Real-time</b><br>SSE, measured 65 ms                                              | IMPLEMENTED |
| <b>SOS — online</b><br>e2e + concurrency suites                          | IMPLEMENTED | <b>Payment</b><br>sandbox only — never a live account                                | IMPLEMENTED |
| <b>SOS — offline</b><br>browser suite drives the whole scenario          | IMPLEMENTED | <b>Fleet</b><br>schema + roles exist; no dedicated UI                                | PARTIAL     |
| <b>Offline storage</b><br>IndexedDB, AES-GCM-256, survives restart       | IMPLEMENTED | <b>Analytics</b><br>ops overview with live counts; no BI layer                       | PARTIAL     |
| <b>Store-and-forward sync</b><br>idempotent; duplicate proven impossible | IMPLEMENTED | <b>Cloud autoscaling</b><br>designed; no cluster                                     | TARGET      |
| <b>Dispatch</b><br>ladder, timeout, busy-exclusion, race-safe            | IMPLEMENTED | <b>Replication / load balancing</b><br>stateless API is the precondition, and is met | TARGET      |

GREEN = code exists, database persists it, UI consumes it, AND a test exercises it. AMBER = some of that. BLUE = architecture only.

# Limitations

In order of importance. Every one of these is also written in the code or the docs.

- 1 Nothing is deployed to a cloud — no account, no domain, no cluster.
- 2 Single instance only: SSE registry, rate limiter and offer sweeper are in-process.
- 3 The ERSS 112 handoff is a stub, and the API response says so. Emergency isolation (ADR-0005) is a design, not a deployment.
- 4 Payments verified against a local stub of Razorpay's API — never a real account.
- 5 The diagnosis “AI” is a deterministic rules engine. Labelled as such everywhere.
- 6 No load test, no external penetration test, no coverage on the HTTP layer (suites run out-of-process).
- 7 Device encryption protects a storage dump, not script on the same origin — and the UI says exactly that.
- 8 GPS cold start can take minutes and may never fix indoors. The app shows “Unknown” rather than a guess.
- 9 No historical volume yet for predictive maintenance — the dead-zone heuristic is labelled heuristic v1.

FUTURE — LABELLED AS SUCH

# Roadmap

## NEXT

- Redis-backed rate limiting
- Outbox → event bus for SSE fan-out
- WAL archiving (RPO under a day)
- Object storage for photos

## THEN

- Multi-region deployment
- Database read replicas
- External APM + error reporting
- Automated backup schedule

## LATER

- Predictive maintenance at volume
- Fleet management UI
- Satellite emergency communication
- Mesh networking between devices
- Government emergency-network integration

Satellite, mesh and government-network integration are FUTURE and are labelled that way everywhere in the product — not implemented, not claimed.

# Where the money would come from

Potential, not current. No commercial operation exists — this is a university project.

## B2C

Subscription for individual drivers; pay-per-incident for non-members

## B2B

Fleet operators — uptime is their cost centre; per-vehicle per-month

## B2B2C

Insurers and OEMs bundling roadside cover into an existing policy

## B2G

Highway authorities — RAKSHA road-health data and incident analytics

The unit economics that make it plausible: the platform runs on ZERO paid third-party accounts by default.  
Maps are keyless OpenStreetMap, AI is a local rules engine, and every vendor sits behind an adapter with a working local implementation.

Current status: no revenue, no customers, no commercial deployment. Stated so the model is read as a plan, not a claim.

# What a competitor would have to build

Not “we are better than X” — we have not benchmarked anyone. This is what is genuinely hard here.

## Network resilience

An emergency workflow that runs entirely on the device, with an encrypted journal and idempotent forward-sync. This is the moat.

## Honest degradation

The product never claims something reached the cloud when it did not. That is a design discipline, not a feature you can add later.

## Race-safe dispatch

Row-locked assignment, wave ladder, timeout escalation, derived provider state — proven with ten simultaneous accepts.

## Explainable diagnosis

The same rule table on device and cloud, with a CI test that fails the build if they diverge.

## Auditability

Hash-chained append-only log, verified live in the operations view.

# Live demonstration

## PART ONE · THE FULL JOURNEY

1. Login
2. Vehicle
3. Breakdown + AI diagnosis
4. Request assistance
5. Dispatch — ranked offers
6. Mechanic accepts
7. Live status, no refresh
8. Service → invoice → payment
9. Review

## PART TWO · THE ONE NOBODY ELSE HAS

1. Network OFF — the UI changes immediately
2. Local diagnosis, labelled
3. Hold SOS → off-grid incident, real GPS
4. "Nothing has been transmitted"
5. CLOSE THE TAB. REOPEN IT.
6. The incident is still there
7. Network ON → automatic sync
8. "SOS synchronized"
9. Replay the reference → duplicate refused

If any step fails, the failure IS the demonstration — nothing here fakes a success.

# RoadAssist Bharat

From breakdown to resolution — even when connectivity fails.

## AI

explainable  
rules engine

## CLOUD

consumed and  
provided

## OFFLINE

the whole emergency  
path on-device

## REAL-TIME

65 ms, and never  
the only path

## SECURITY

74 attacks,  
all refused

## DISPATCH

dynamic, race-safe  
scheduling

“RoadAssist doesn't stop when the network stops.”

631 assertions · 6 suites · 0 failures · 56 tables · 64 routes · 10 ADRs · every claim on these slides is testable